What happens in JavaScript when you assign a value to a variable that has not been explicitly scoped to the function with `var`? The variable is declared globally *for you*.[3] See chapter 13. This is a massive problem in a browser environment where scripts from many sources that have potentially never been tested together are loaded into a single webpage sharing a single global scope.

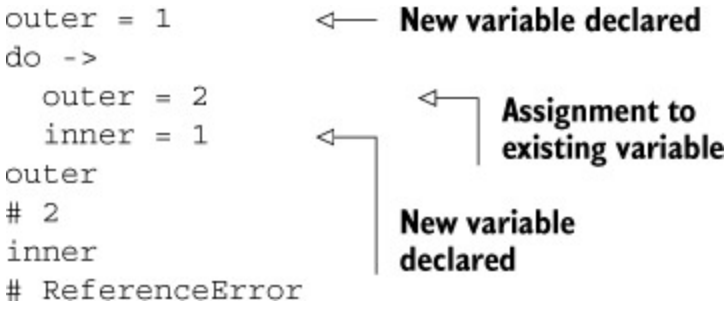
[3] The fifth edition of the ECMAScript specification can help with this problem in JavaScript.

It was one thing for Robin to accidentally clobber your variable; it's another thing for a website running your script to load another script that accidentally or even deliberately clobbers your variables. The previous example would be written in CoffeeScript as

```
scope = ->
  x = 1
  y = 2
```

The variables will be defined in the current function scope implicitly. It's not possible to create a global variable unless you're explicitly trying to.

CoffeeScript variables don't require a `var` keyword; when an assignment to a variable is made, the name is first looked for in the current scope. If it isn't found, then it's created in the current scope. Because it isn't possible to create a new local variable that already exists in an outer scope, CoffeeScript is said to not have variable *shadowing*:



Implicit variable definition in CoffeeScript means that variables can't accidentally be defined on the global scope. CoffeeScript goes one step further, though; in order to stop variables being placed in the global scope at